Ejercicio 2: Automatización con GitHub Actions

Este ejercicio extiende el trabajo anterior automatizando las pruebas cada vez que subes cambios al repositorio (CI/CD). Aquí simularemos el flujo real de trabajo: subir código roto, ver que el CI falla, arreglarlo, y ver que el CI pasa.

Estructura

Idealmente, este folder representa la raíz de un repositorio git.

- .github/workflows/validate_dags.yml: El archivo de configuración para GitHub Actions.
- dags/: Tus DAGs.
    - good_dag.py: DAG correcto.
    - bad_dag.py: DAG con ciclo (error).
- tests/: Tus pruebas automatizadas.

Pasos para simular el Ciclo de CI/CD

1. Setup inicial de Git y GitHub

1. Inicializa git en esta carpeta:

```
git init
git branch -M main
```

2. Crea un repositorio **VACÍO** en tu cuenta de GitHub.
3. Conecta tu repositorio local con el remoto:

```
git remote add origin <URL_DE_TU_REPO>
```

2. Subir código con ERROR (El "Bad DAG")

Actualmente tienes bad_dag.py que contiene un ciclo. Vamos a subirlo para ver cómo GitHub Actions nos protege.

```
git add .
git commit -m "feat: adding dags with potential errors"
git push -u origin main
```

3. Verificar el Fallo en GitHub Actions

1. Ve a la pestaña **Actions** en tu repositorio de GitHub.
2. Verás el workflow ejecutándose.
3. **Resultado**: El workflow fallará.
4. Entra en el detalle del paso "Ejecutar Tests con Pytest" y verás el error: Cycle detected in DAG: bad_cyclic_dag.

4. Corregir y Subir (El "Fix")

Ahora actuaremos como desarrolladores responsables corrigiendo el error.

1. Elimina el archivo malo localmente:

```
rm dags/bad_dag.py
```

2. Sube la corrección:

```
git add .
git commit -m "fix: removing broken dag logic"
git push
```

5. Verificar el Éxito

1. Vuelve a la pestaña **Actions**.
2. Verás un nuevo workflow ejecutándose por el nuevo commit.
3. **Resultado**: Esta vez, el workflow terminará en verde.